SLE-3: ARCHITECTURAL DESIGN USING FULL C4 MODEL

Course: 02AML204 – Introduction to Artificial Intelligence

Program: SY B.Tech. CSE (AI & ML)

Student Name:	Tanishka Vijay Chougule
PRN:	25UAM086
Division:	B
Date:	03/10/2026
System Title:	BFS vs DFS – Maze Path-Finding Profiling

1. System Title and Short Description

System Title: BFS vs DFS – Maze Path-Finding Profiling

The BFS vs DFS Maze Path-Finding system compares Breadth-First Search (BFS) and Depth-First Search (DFS) on randomly generated grid mazes. The system finds paths from a start point to a goal and measures execution time, nodes expanded, and path length. The results are used to study and compare the behaviour of the two search algorithms.

2. Context Diagram

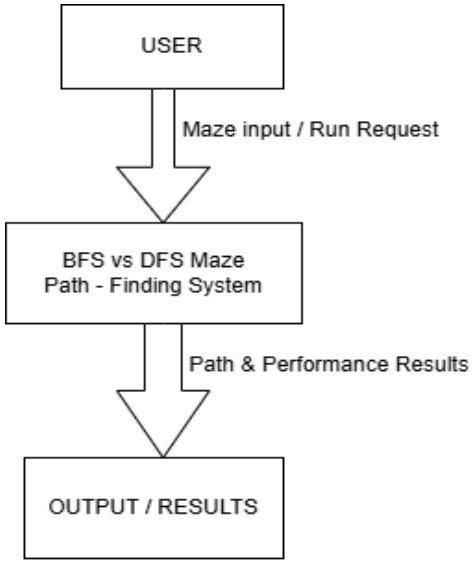


Figure 1: Level 1 – Context Diagram

Explanation: The BFS vs DFS Maze Path-Finding System takes a maze input and performs path finding using BFS and DFS algorithms. The system processes the maze and searches for a path from the starting point to the goal. It produces the paths and performance results such as execution time, nodes expanded, and path length.

3. Container Diagram

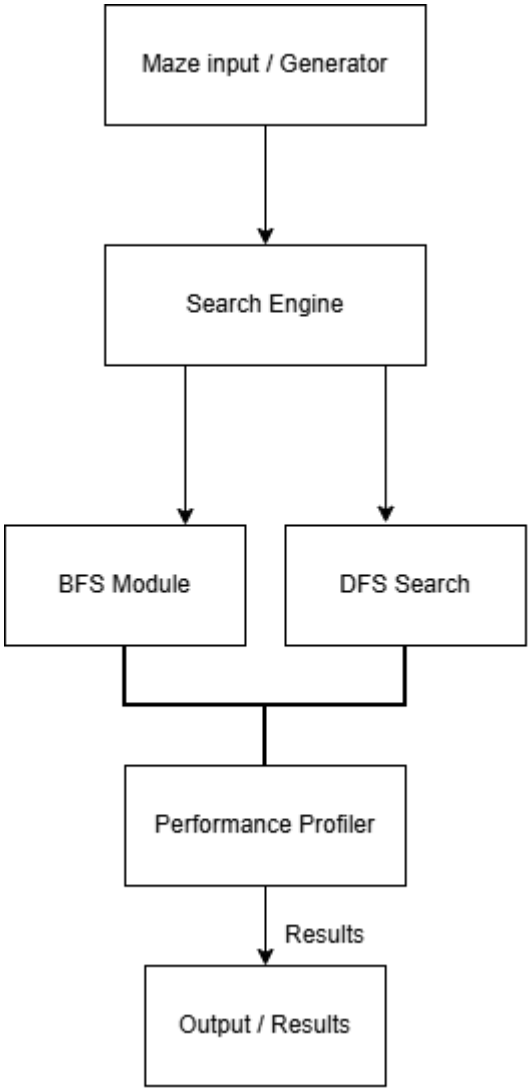
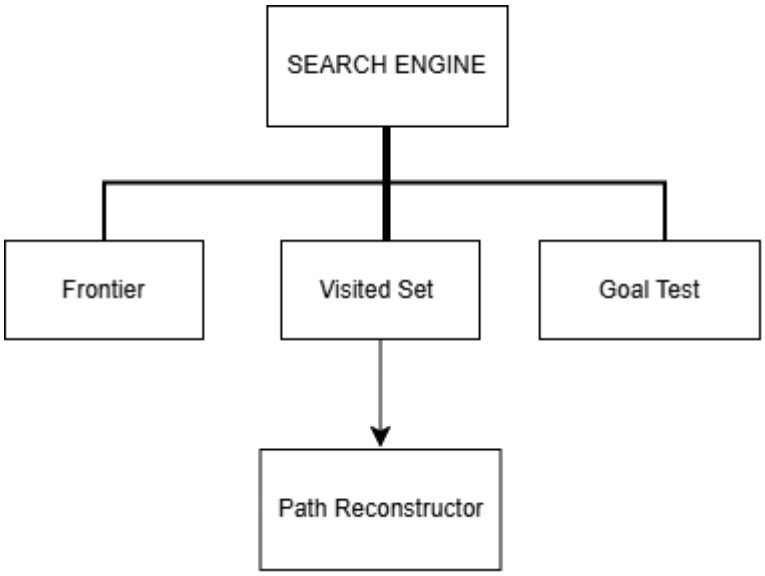


Figure 2: Level 2 – Container Diagram

3. Container Diagram – Container Descriptions

1. **Maze Input / Generator:** Generates or accepts the maze used for path finding.
2. **Search Engine:** Controls the search process and coordinates the path-finding operation.
3. **BFS Module:** Performs Breadth-First Search on the maze.
4. **DFS Module:** Performs Depth-First Search on the maze.
5. **Performance Profiler:** Measures execution time and records performance information such as nodes expanded and path length.
6. **Output / Results:** Presents the paths and profiling results for comparison.

4. Component Diagram



Explanation: The Search Engine is divided into smaller components that support the search process. The Frontier stores the cells waiting to be explored, while the Visited Set keeps track of already explored cells. The Goal Test checks whether the goal has been reached. The Path Reconstructor builds the final path from the search information.

5. Code Level Overview

The main functions used in the BFS vs DFS Maze Path-Finding system are:

1. **generate_maze()** – Generates a random maze with walls and open paths.
2. **get_neighbors()** – Finds valid neighbouring cells for a given cell.
3. **bfs()** – Performs Breadth-First Search to find a path from the start to the goal.

4. **dfs()** – Performs Depth-First Search to find a path from the start to the goal.

5. **reconstruct_path()** – Reconstructs the final path after the search is completed.

6. **time.perf_counter()** – Measures the execution time of the algorithms.

7. **cProfile** – Profiles the program to study its performance.

6. Design Decisions

The architecture separates maze generation, search algorithms, profiling, and output so that each part has a clear responsibility. BFS and DFS are kept as separate modules because their search strategies and data structures are different. The Search Engine is selected for the component-level view because it is the main part responsible for coordinating the search process.

7. AI Contribution Note

AI tools were used to support the architectural design process, understand the C4 model levels, organize the system into Context, Container, Component, and Code views, and improve the clarity of the written explanations. The final diagrams and content were reviewed and arranged according to the student's SLE-2 project and SLE-3 requirements.

8. Conclusion

The Full C4 Model provides a structured view of the BFS vs DFS Maze Path-Finding system. The Context, Container, Component, and Code levels show the system from a high-level view to its main functions. This architecture makes the relationship between the maze input, search algorithms, performance profiling, and output easier to understand.0